WEEK 1 · LESSON 5 OF 24 · HANDOUT

PyTorch: Tensors and Autograd

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=ORMx45xqWkA — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython: An introduction to PyTorch and autograd — https://www.youtube.com/watch?v=rrekAv9Fml4

🎓 **Go deeper:** freeCodeCamp: PyTorch for Deep Learning & Machine Learning — full course — https://www.youtube.com/watch?v=V_xro1bcAuA

Your goal in this lesson: Create and reshape PyTorch tensors, use a GPU when one is available, let autograd compute gradients, and rebuild last lesson's network in a few lines.

Why PyTorch?

PyTorch is the most used deep-learning library in research and industry (with TensorFlow/Keras and JAX as the others). It gives you:

1. **Tensors** — like NumPy arrays, but able to run on a GPU.
2. **Autograd** — automatic backpropagation: you write the forward pass, PyTorch computes every gradient.
3. **Ready-made building blocks** — layers, losses, optimisers and data tools.

It is already installed in Google Colab. To get a free GPU there: **Runtime → Change runtime type → T4 GPU**.

Tensors

```
import torch

a = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
print(a.shape, a.dtype)
print(a * 10)               # element-wise, like NumPy
print(a @ a)                # matrix multiplication
print(a.mean(), a.sum(dim=0))

z = torch.zeros(3, 4)
r = torch.randn(2, 3)       # random normal numbers
print(z.shape, r.shape)
print(r.reshape(3, 2).shape, r.T.shape)
```

Moving between NumPy and PyTorch is easy:

```
import numpy as np

arr = np.arange(6).reshape(2, 3)
t = torch.from_numpy(arr).float()   # NumPy -> tensor (float32 for networks)
back = t.numpy()                    # tensor -> NumPy
print(t, back.dtype)
```

Neural networks use **float32** numbers almost everywhere — that is why we call `.float()`.

CPU or GPU?

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using:", device)
x = torch.randn(1000, 1000, device=device)
y = x @ x          # runs on the GPU if there is one
print(y.shape, y.device)
```

Write `device` once at the top of every notebook and move your model and data to it with `.to(device)`. The same code then runs on a laptop CPU or a Colab GPU.

Autograd: gradients for free

Mark a tensor with `requires_grad=True` and PyTorch records every operation on it. Calling `.backward()` runs backpropagation:

```
w = torch.tensor(3.0, requires_grad=True)
loss = (w - 5) ** 2          # smallest when w = 5
loss.backward()
print("gradient:", w.grad)   # 2 × (3 − 5) = −4
```

The gradient is **−4**: increasing w lowers the loss, exactly what the derivative rules give. Let gradient descent find w = 5:

```
w = torch.tensor(0.0, requires_grad=True)
for step in range(50):
    loss = (w - 5) ** 2
    loss.backward()
    with torch.no_grad():      # updating weights is not part of the graph
        w -= 0.1 * w.grad
    w.grad.zero_()             # gradients add up unless you reset them!
print(round(w.item(), 3))
```

Two rules to remember: update weights inside `torch.no_grad()`, and **reset gradients** every step — PyTorch adds new gradients to old ones by default.

Last lesson's network, with autograd

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

X, y = make_moons(n_samples=1000, noise=0.2, random_state=0)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
Xtr, ytr = torch.tensor(X_train, dtype=torch.float32), torch.tensor(y_train,
dtype=torch.float32)
Xte, yte = torch.tensor(X_test, dtype=torch.float32), torch.tensor(y_test, dtype=torch.float32)

torch.manual_seed(0)
W1 = (torch.randn(2, 16)).requires_grad_()
b1 = torch.zeros(16, requires_grad=True)
W2 = (torch.randn(16, 1) * 0.25).requires_grad_()
b2 = torch.zeros(1, requires_grad=True)
params = [W1, b1, W2, b2]

for epoch in range(2000):
    hidden = torch.relu(Xtr @ W1 + b1)
    logits = (hidden @ W2 + b2).squeeze(1)
    loss = torch.nn.functional.binary_cross_entropy_with_logits(logits, ytr)
    loss.backward()                          # all four gradients, automatically
    with torch.no_grad():
        for p in params:
            p -= 0.5 * p.grad
            p.grad.zero_()

with torch.no_grad():
    pred = torch.relu(Xte @ W1 + b1) @ W2 + b2
    acc = ((pred.squeeze(1) > 0) == yte.bool()).float().mean()
print("loss:", round(loss.item(), 3), " test accuracy:", round(acc.item(), 3))
```

There is **no backward code** — `loss.backward()` replaces the six chain-rule lines from Lesson 4, and the network again reaches about **0.96**.

`binary_cross_entropy_with_logits` combines the sigmoid and the loss in one numerically safe step, so the model outputs raw scores (logits). A logit above 0 means a probability above 0.5.

Key terms

- **PyTorch** — a deep-learning library built on tensors and autograd.
- **Tensor** — a multi-dimensional array that can live on a CPU or GPU.
- **Autograd** — PyTorch's automatic differentiation (backpropagation).
- **requires_grad** — tells PyTorch to track a tensor for gradients.
- **torch.no_grad()** — switch off tracking, for updates and predictions.
- **Device** — where computation runs: "cpu" or "cuda" (GPU).

Remember this: PyTorch tensors are GPU-ready NumPy arrays, and autograd turns any forward computation into gradients with one call to `.backward()`.

Name: _____ Date: _____

Activity

Task 1

Create a tensor of shape (2, 3, 4). What are `t.shape[1]` and `t.numel()`?

Task 2

Remove `p.grad.zero_()` from the moons loop. What happens to training, and why?

Task 3

Use autograd to find the slope of $y = x^3$ at $x = 2$. Check it with the derivative rule ($3x^2$).

Task 4

In Colab, switch to a T4 GPU and time a (4000 × 4000) matrix multiplication on CPU and GPU.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is the main extra power of a PyTorch tensor compared with a NumPy array?
   - A. It cannot do maths
   - B. It can run on a GPU and track gradients
   - C. It can only hold text
   - D. It is always smaller
2. What does loss.backward() do?
   - A. Splits the data
   - B. Computes gradients for every tensor that requires them
   - C. Deletes the loss
   - D. Updates the weights
3. Why call grad.zero_() every step?
   - A. To delete the model
   - B. PyTorch adds new gradients to old ones unless you reset them
   - C. It is not needed
   - D. To speed up the GPU
4. For loss = $(w - 5)^2$ at $w = 3$, what gradient does autograd give?
   - A. 25
   - B. 0
   - C. −4
   - D. 4
5. Which number type do neural networks usually use?
   - A. string
   - B. bool
   - C. int8 only
   - D. float32
6. Why update weights inside torch.no_grad()?
   - A. It saves the model
   - B. It makes the update bigger
   - C. To use the CPU
   - D. The update itself should not be recorded as part of the model's computation

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.